

EECS 482

Introduction to operating systems

Client-server systems

Peter Chen
University of Michigan

Client-server

- Common way to structure a distributed application:
 - **Server** provides some centralized service
 - **Client** makes request to server, waits for response
- Example: **web server**
 - Server stores and returns web pages
 - Clients run Web browsers, make GET/POST requests
- Example: **coke machine**
 - Server manages state associated with coke machine
 - Clients call `client_produce()` or `client_consume()`, which send request to the server and return when done
 - Server responds to client after request has been satisfied

Client-server example

```
client_produce() {  
    send produce request to server  
    receive response  
}
```

```
server() {  
    while (true) {  
        receive request  
        if (produce request) {  
            add coke to machine  
        } else {  
            ...  
        }  
        send response  
    }  
}
```

Problems?

Example with waiting

```
server() {  
  while (true) {  
    receive request  
    lock  
    if (produce request) {  
      while (machine is full) wait  
      add coke to machine  
      signal  
    } else {  
      ...  
    }  
    send response  
    unlock  
  }  
}
```

When to create threads?

Example with threads

```
server() {  
    while (true) {  
        receive request  
        if (produce request) {  
            create thread that calls server_produce()  
        } else {  
            create thread that calls server_consume()  
        }  
    }  
}
```

Problems?

```
server_produce() {  
    lock  
    while (machine is full) wait  
    add coke to machine  
    signal  
    send response  
    unlock  
}
```

Example with threads

```
server() {  
    while (true) {  
        wait for new connection from client  
        create thread that calls handle_request()  
    }  
}
```

```
handle_request() {  
    receive request  
    call server_produce() or server_consume()  
}
```

What are the slow operations?

```
server_produce() {  
    lock  
    while (machine is full) wait  
    add coke to machine  
    signal  
    send response  
    unlock  
}
```

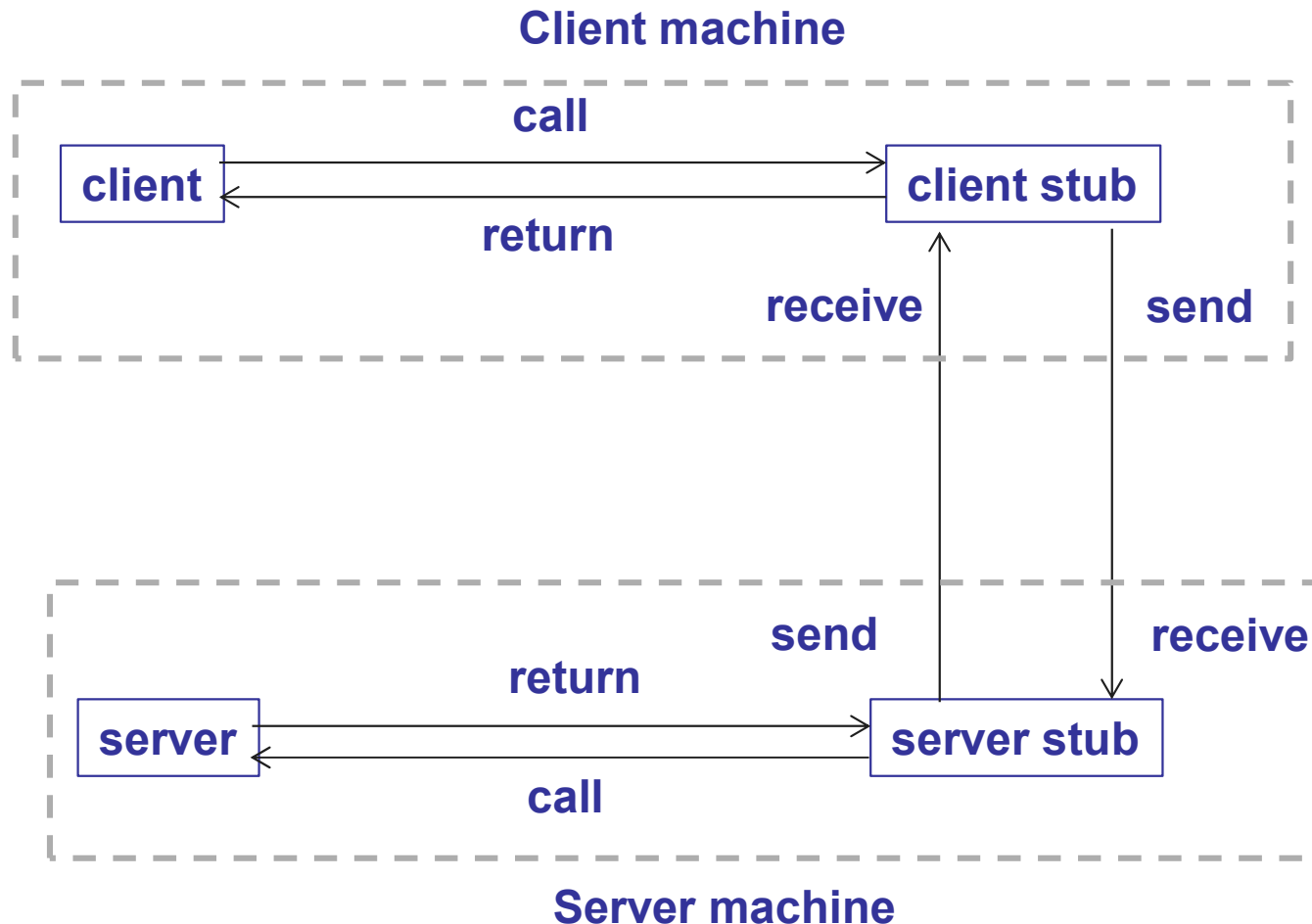
Alternatives

- How to lower overhead of creating threads?
- How to structure the server to avoid blocking for slow operations
 - Synchronous/blocking: call slow operation and wait for it
 - » Threads
 - Asynchronous/non-blocking: don't wait for slow operation
 - » Polling (via `select` or `poll`)
 - » Callbacks
- What to do if there's no work?

Client-server interface

- Message send/receive
- **Alternative?**

RPC abstraction via stub functions on client and server



Producer-consumer using RPC

Client stub

```
int produce (int n) {  
    int status;  
    → send (sock, &n, sizeof(n));  
    → recv (sock, &status, sizeof(status), MSG_WAITALL);  
    → return(status);  
}
```

Server stub

```
void produce_stub () {  
    int n;  
    int status;  
    → recv (sock, &n, sizeof(n), MSG_WAITALL);  
    → status = produce(n);  
    → send (sock, &status, sizeof(status));  
}
```

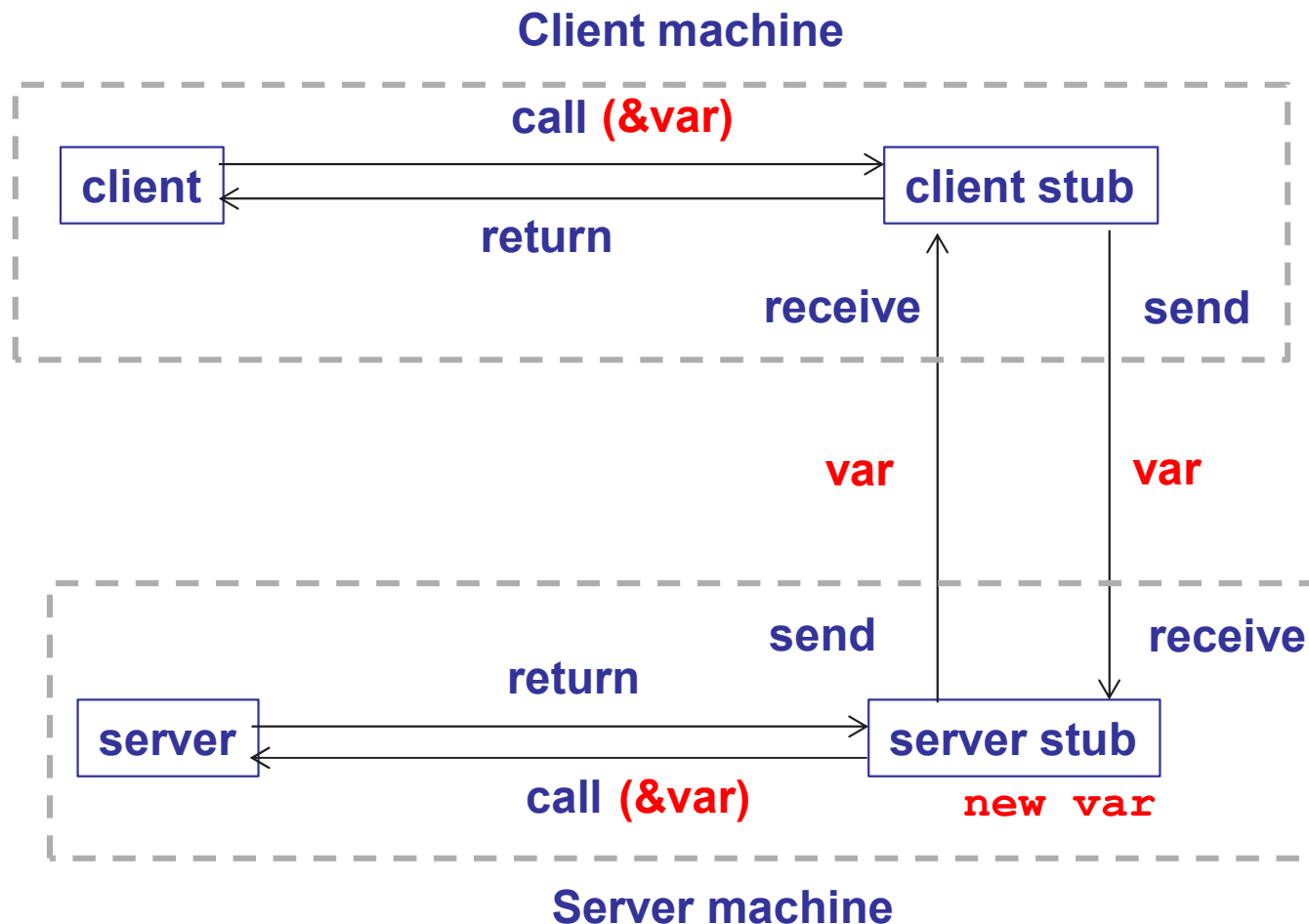
Generation of stubs

- Stubs can be generated automatically
- What do we need to know to do this?
- Interface description:
 - Types of arguments and return value
- e.g. rpcgen on Linux

RPC transparency

- RPC makes send/receive to server look like local procedure calls
 - Basis of CORBA, Thrift, SOAP, Java RMI, AJAX, ...
- What factors break illusion?

How to pass a pointer to remote system?



Data representation

- Client and server must agree on how to represent data
- E.g., endianness of multi-byte integers
 - In a 4-byte integer, which is the least significant byte?

Byte 3	DE
Byte 2	AD
Byte 1	BE
Byte 0	EF

Is this 0x DE AD BE EF or
0x EF BE AD DE?

- E.g., character representation
- E.g., C-string

Failures

- How are failures in a distributed system different from failures on a local system?
- When have you used RPC in EECS 482?

Project 4

- Use `showfs` to verify that contents of file system match your expectations
- Use `assertions` to catch errors early
 - # of free disk blocks matches file system contents?
 - `showfs` has many built-in assertions. Try running `showfs` from the file server.
- Testing
 - Cover all states of data structures (e.g., `direntries` array)
 - Test boundaries (cf. constants in `fs_server.h`, `fs_param.h`)

From operating systems to distributed systems

- Why build distributed systems?
- People are distributed
 - Web, e-mail, SMS, social media, Google docs, Zoom
- Performance
 - Aggregate performance of many computers can be faster than that of (even a fast) single computer
- Reliability
 - Try to provide continuous service, even if some computers fail
 - Try to preserve data, even if some storage fails

Distributed systems are concurrent

- Concurrent programs need mechanisms for:
 - Synchronizing
 - Sharing information
- How did we accomplish these in multi-threaded programs?
- How should we accomplish these in a distributed system?

Using send/receive to share information

- Two styles of sharing information
 - Shared memory
 - Message passing

How to structure the server to avoid blocking for slow operations

- Synchronous/blocking: call slow operation and wait for it
 - Threads
- Asynchronous/non-blocking: don't call slow operation
 - Polling (via `select` or `poll`): check before calling
 - Callbacks: ask slow operation to interrupt you
- What to do if there's no work?

Using send/receive to synchronize

- How to provide mutual exclusion?
 - Need atomic primitive
 - In a distributed system, that atomic primitive is sending/receiving a network packet
 - How to build a larger critical section, e.g., sending multiple messages?
- How to provide ordering?
- Without shared data, are race conditions still possible?

Structuring a concurrent system

- One multi-threaded process on one computer
- Several multi-threaded processes, each running on a separate computer
- Several multi-threaded processes, all running on one computer.
 - Why do this?
 - Operating systems structured like this are called **microkernels**

Computer 1
Thread A
Thread B

Computer 1
Thread A
Thread B

Computer 2
Thread A
Thread B

Computer 1	
Process 1	Process 2
Thread 1A	Thread 2A
Thread 1B	Thread 2B